

Proyecto Final de Bases de Datos: Amazon Reviews

Jorge Ois de Pascual · Miguel Pajuelo Gomez | Grado en Ingeniería Matemática e Inteligencia Artificial (iMAT) | Bases de Datos

4 datasets Toys & Games, Video Games, Digital Music y Musical Instruments	3 gestores MySQL + MongoDB + Neo4j	7 scripts carga, visualización, Neo4j, inserción, configuración y requisitos	8 librerías utilizadas pymysql, pymongo, matplotlib, seaborn, wordcloud, neo4j, networkx y pandas
---	--	--	---

DESCRIPCIÓN

Hemos diseñado un sistema mixto (MySQL + MongoDB) que permite almacenar datos estructurados, texto libre y estructuras relacionales (Neo4J). Para ello hemos utilizado librerías clave de la base de datos de Amazon, con un total de 474.344 reviews distribuidas en cuatro categorías: Toys & Games (167.597), Video Games (231.780), Digital Music (64.706) y Musical Instruments (10.261).

APORTACIÓN TÉCNICA

- Inserción de datos en el sistema sin sobrecarga en memoria
- Separación razonada entre datos estructurados (MySQL) y texto libre (MongoDB).
- Modelado relacional con trazabilidad: review → artículo → categoría.
- Extensión analítica con Neo4j para estudiar similitudes entre usuarios y grafos de consumo.

4.6. Flujo general del pipeline



Figura 2: Pipeline general del proyecto

Figura 1. Pipeline del proyecto: parseo JSON, separación estructurado/texto y explotación analítica.

DESARROLLO Y DIVISIÓN DE DATOS

La solución se organiza en siete ficheros principales: *configuracion.py*, *load_data.py*, *menu_visualizacion.py*, *neo4jProyecto.py*, *insertar_dataset.py*, *apartado_6_2.tex* y *requirements.txt*.

MySQL almacena *reviewerID*, *reviewerName*, *asin*, *categoría*, *overall*, fechas y utilidad;

MongoDB guarda *reviewText* y *summary* enlazados por *reviewerID* y *asin*;

Neo4j modela usuarios, artículos y relaciones derivadas.

VISUALIZACIÓN Y CONSULTAS

- Menú Python que muestra: evolución temporal, popularidad de artículos, histograma por nota, evolución acumulada, histograma por usuario, nube de palabras y boxplot por categoría.
- Apoyo externo con Power BI para reforzar tres gráficas del dataset.
- Recomendación top-10 por popularidad condicionada al historial de consumo del usuario.

El fichero *menu_visualizacion.py* implementa una aplicación de consola que mantiene un bucle interactivo hasta que el usuario selecciona la opción de salida. El menú final integra todos apartados de visualización del enunciado así como la funcionalidad de recomendación pedida al final del proyecto.

```
print("1. Evolución de reviews por años")
print("2. Popularidad de artículos")
print("3. Histograma por nota (overall)")
print("4. Evolución temporal acumulada")
print("5. Histograma de reviews por usuario")
print("6. Nube de palabras (Summary)")
print("7. Gráfico extra (Boxplot por categoría)")
print("8. Funcionalidad apartado 6.3: recomendar 10 artículos no consumidos")
print("9. Salir")
```

A nivel funcional, el programa se conecta simultáneamente a MySQL y MongoDB, y según la opción elegida consulta una u otra base de datos, o ambas.

Figura 2. Fragmento del menú de la aplicación de visualización y recomendación.

RESULTADOS

El análisis revela una distribución sesgada hacia las puntuaciones más altas, especialmente las de 4 y 5 estrellas, esto viene causado por una estructura "longtail"; unos pocos artículos condensan gran parte de las reviews (principalmente positivas de ahí la asimetría) y la mayoría de reciben pocas reseñas.

La parte relacional de Neo4J nos ha permitido identificar similitudes entre usuarios o patrones de consumo que no se ven a simple vista con MySQL o MongoDB

CONCLUSION Y CIERRE

El proyecto cubre modelado, ingestión, persistencia, consulta, visualización, análisis relacional y una primera capa de recomendación en una arquitectura reutilizable y extensible.

Esta visualización ilustra con claridad la *long tail* del catálogo: unos pocos *asin* concentran gran parte de las interacciones, mientras que la mayoría queda muy por debajo.

7. Aplicación Python con Neo4j

El fichero *neo4jProyecto.py* extiende el proyecto hacia un análisis relacional. En lugar de pensar solo en tablas, en este apartado hemos desarrollado grafos a partir de las interacciones almacenadas en MySQL. El script define parámetros fácilmente modificables como se solicitaba en

```
TOP_USUARIOS_SIMILITUD = 30
PRIMEROS_USUARIOS = 400
TOP_ARTICULOS_POPULARES = 5
MAX_REVIEWS_ARTICULO_POPULAR = 40
```

7.1. Apartado 4.1: similitudes entre usuarios

7.1.1. Idea del apartado

Se seleccionan los 30 usuarios con más reviews, se recuperan sus artículos puntuados y se calcula la correlación de Pearson entre cada par. Solo se almacenan similitudes entre usuarios que comparten al menos un artículo.

Figura 3. Parámetros y planteamiento del módulo analítico con Neo4j.